Arjun Arvind Gahane (25070521135)

Exp 15.1

Algorithm:

Step 1: Define the Blueprint (Base Class)

- Create a class named Car.
- Initialize it with four parameters: brand, price, model, and color.
- Store these parameters as **instance variables** so every car object knows its own data.

Step 2: Establish Inheritance (Derived Classes)

- Create two new classes, Car1 and Car2.
- Link them to the Car class. This ensures they "inherit" the ability to store the brand, price, model, and color without needing new code to handle that data.

Step 3: Implement Method Overriding

- Inside Car1, define a method display_details().
- Inside Car2, define a method display_details().
- Even though they have the same name, each method is responsible for printing its specific object's data in the required four-line format.

Step 4: Receive and Parse Input

- Read the first line of raw input from the user.
- **Split** that string into a list of four separate items.
- Convert the "price" item into a float (numeric) type.

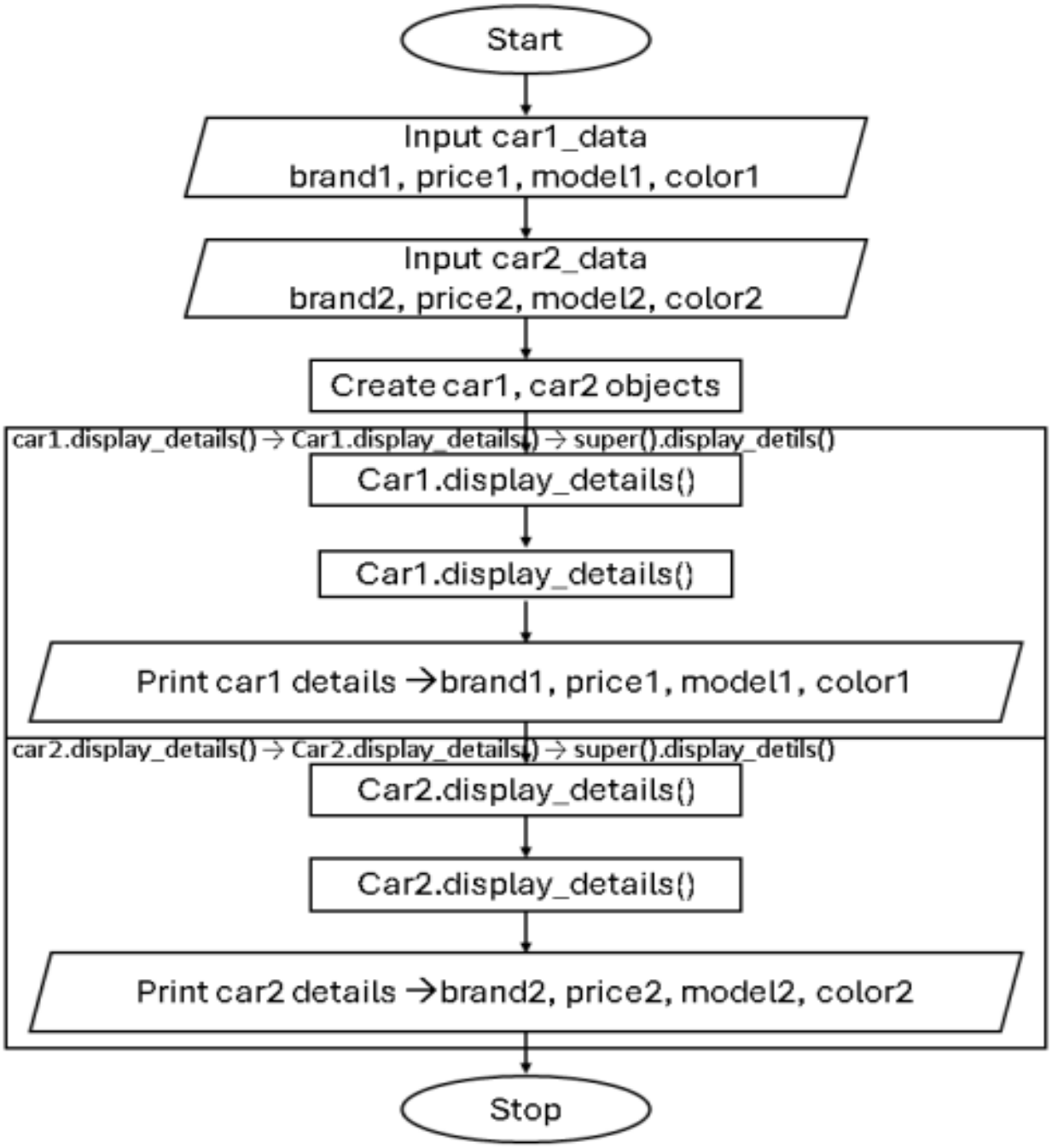
Step 5: Object Instantiation

- Pass the first set of data into Car1 to create an "instance" (a specific car object).
- Repeat the process for the second line of input to create an instance of Car2.

Step 6: Output Generation

- Trigger the display_details() method for the first car.

Flowchart :


Start
Input car1_data
brand1, price1, model1, color1
Input car2_data
brand2, price2, model2, color2
Create car1, car2 objects
car1.display_details() → Car1.display_details() → super().display_detils()
Car1.display_details()
Car1.display_details()
Print car1 details →brand1, price1, model1, color1
car2.display_details() → Car2.display_details() → super().display_detils()
Car2.display_details()
Car2.display_details()
Print car2 details →brand2, price2, model2, color2
Stop

Code:

CODETANTRA Home

arjun.gahane.batch2025@sitnagpur.siu.edu.in Support Logout

15.1.1. Car Dealership Sales System 08:29

Write a Python program to model a car dealership sales system using object-oriented programming. Create a base class **Car** and two derived classes **Car1** and **Car2** to represent different car types.

Class Structure

1. Base Class **Car**:
   - Attributes: **brand**, **price**, **model**, **color.**
   - Method: **display_details()** to print all car attributes.
2. Derived Classes **Car1** and **Car2**:
   - Inherit from **Car** class.
   - Override **display_details()** method to display car information in the specified format.

Input Format:

- The first line contains **Car1**'s **brand** (string), **price** (float), **model** (string), and **color** (string) separated by spaces.
- The second line contains **Car2**'s **brand** (string), **price** (float), **model** (string), and **color** (string) separated by spaces.

Output Format:

- First four lines: **Car1** details (**brand**, **price**, **model**, **color** - each on separate line)
- Next four lines: **Car2** details (**brand**, **price**, **model**, **color** - each on separate line)

Sample Test Cases

Test case 1

Explorer

carDetail... Submit

```python

# Base class
class Car:
    def __init__(self, brand, price, model, color):
        self.brand = brand
        self.price = price
        self.model = model
        self.color = color

    def display_details(self):
        print(self.brand)
```

Debugger

Average time	Maximum time
0.010 s	0.014 s
9.83 ms	14.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 14 ms Debug

Expected output	Actual output
Toyota 25000.50 Camry White	Toyota 25000.50 Camry White
Honda 22000.00 Accord Black	Honda 22000.00 Accord Black
Toyota	Toyota
25000.5	25000.5
Camry	Camry
White	White

Terminal Test cases

Prev Reset Submit